ENTERPRISE RESTAURANT & MULTI-OUTLET SYSTEM

Complete End-to-End Development & Redesign Blueprint

Production Architecture, Database Design, Modular Feature Engineering & Step-by-Step Re-Creation Guide

Comprehensive Developer Edition • Full Lifecycle Reference

PHASE 1: FOUNDATION, RUNTIME & ENVIRONMENT CONFIGURATION

1.1 Initial Project Setup & Tooling Selection

When building an interactive restaurant website that incorporates live ordering, a kitchen display system (KDS), and in-venue gaming, we choose Vite + React 19 + TypeScript. Vite ensures sub-second Hot Module Replacement (HMR) and optimized Rollup tree-shaking for mobile devices on venue Wi-Fi.

- **Node Version:** Node.js 20+ LTS recommended for native fetch and ESModule support.
- **TypeScript Strictness:** Strict mode enabled (noImplicitAny, strictNullChecks) to guarantee rock-solid order pricing.
- **Tailwind CSS v4:** Zero-runtime utility engine utilizing modern CSS variables for dark luxury theme.

TERMINAL COMMANDS

```
npm create vite@latest restaurant-system -- --template react-ts
cd restaurant-system
npm install -D @types/canvas-confetti @types/qrcode @types/node
```

PHASE 2: FEATURE-DRIVEN DIRECTORY HIERARCHY

2.1 Why Feature Folders Beat Traditional Type Folders

In a basic student tutorial, developers place all components inside "components/". Once the application grows with game lounges, kitchen dashboards, cart modals, and restaurant brand switchers, that folder becomes unmaintainable. Instead, we use the Domain Feature Pattern where each capability encapsulates its own UI, data, and helpers.

DIRECTORY TREE

```
src/
% % %  features/
%    % % %  games/            # Self-contained games registry & 9 interactive modules
%    %    % % %  components/   # SpinWheel, Snake, MemoryMatch, Uno, Ludo, etc.
%    %    % % %  gamesRegistry.ts
%    %    % % %  GamesLounge.tsx
%    % % %  menu/             # Dishes, dietary pills, live search, customization modal
%    %    % % %  components/   # MenuCatalog, MenuItemCard, CategoryNav, DishCustomizerModal
%    %    % % %  index.ts
%    % % %  cart/             # Tray calculations, GST tax, Dine-In vs Parcel checkout
%    %    % % %  components/CheckoutModal.tsx
%    % % %  orders/           # Live lifecycle tracker & table seating status
%    %    % % %  components/ActiveOrderTracker.tsx
%    % % %  kitchen/          # Kitchen Display System (KDS) for kitchen cooks
%    %    % % %  components/KitchenDashboard.tsx
%    % % %  admin/            # Outlet management, dish CRUD, games toggling, analytics
%    %    % % %  components/AdminDashboard.tsx
%    % % %  restaurants/      # Multi-restaurant brand switcher modal & configurations
%    % % %  table/            # Table QR simulation & seating context manager
%    % % %  lucky-draw/       # Receipt scratch coupon & gamified promotion modal
%    % % %  notifications/    # Kitchen alerts drawer & audio notification pings
% % %  context/CafeContext.tsx
% % %  types.ts
% % %  App.tsx
% % %  main.tsx
```

PHASE 3: MULTI-RESTAURANT FRANCHISE ENGINE & DATA MODEL

3.1 The Multi-Tenant Restaurant Profile

To support other restaurants seamlessly, the entire platform reads from a dynamic `currentRestaurant` object rather than hardcoding names or menus. Each restaurant configures its own address, ratings, GST tax percentage, Wi-Fi credentials, table count, and enabled games.

DATA MODEL (src/types.ts)

```
export interface RestaurantProfile {
  id: string;               // Unique slug: 'roast-toast', 'spice-symphony'
  name: string;             // Display name (e.g. 'The Roast & Toast Artisan Bistro')
  tagline: string;
  cuisine: string;          // e.g. 'Gourmet Burgers & Wood-Fired Pizza'
  address: string;
  rating: number;
  totalReviews: number;
  heroBanner: string;       // High-resolution photo
  logo: string;             // Logo symbol or badge
  themeColor: string;       // Hex accent (e.g. '#f59e0b')
  tableCount: number;
  products: Product[];      // Dedicated menu catalog for this branch
  enabledGameIds?: string[];// Enabled mini-games per location
  isActive?: boolean;
}
```

PHASE 4: MENU ENGINEERING & DIETARY FILTERING

Phase Goal: Build a dynamic menu with dietary badges, customization modals, spice levels, and portions.

4.1 Interactive Menu Customization Modal

Dishes like Gourmet Burgers or Artisanal Pizzas require size choices, crust types, and extra cheese upgrades. The DishCustomizerModal calculates price additions in real-time, validates mandatory choices, and saves chef cooking instructions directly to the item ticket.

- **Diets & Badges:** Filter pills for Pure Veg (emerald), Non-Veg (rose), Chef Special (amber), and Under 15 Mins.
- **Custom Add-ons:** Extra cheese bursts (+ ₹40), gourmet dips (+ ₹25), and portion sizes with instant subtotal.
- **Persistent Floating Cart:** A sleek bottom floating bar that displays quantity and subtotal, guiding users to checkout.

PHASE 5: DINE-IN QR SEATING & PARCEL CHECKOUT

Phase Goal: Support table-based ordering, automatic QR detection, and takeaway packaging.

5.1 Table QR Architecture

Customers seated at a table scan a printed QR stand (`/?table=4`). The website parses the query parameter, binds the table number to the session, and skips manual table entry. If all tables are full, the system automatically routes the diner into the Table Queue while the kitchen prepares their order in parallel.

- **Payment Flow:** Supports Instant Online UPI / Card or Pay at Counter (Cash/Card) with split invoicing.
- **GST Tax Engine:** Dynamic tax rate calculated from restaurant system settings (defaults to standard 5%).

PHASE 6: EXTENSIBLE GAMING LOUNGE & PROMOTIONAL REWARDS

Phase Goal: Engage customers with mini-games while they wait for their food.

6.1 Modular Registry Pattern for Games

Every game in the platform is isolated under `src/features/games/components/`. The central `gamesRegistry.ts` acts as a catalog with metadata: difficulty, player count, rules, and reward discounts.

- **Spin & Win Wheel:** Gives diners a spin during prep time to win free brownies or ₹50 discount vouchers.
- **Retro Café Snake:** Classic touch-enabled arcade game where the snake gobbles hot coffees and pastries.
- **Memory Match:** Food pairing card game with move tracking and visual confetti on victory.
- **Culinary Trivia:** Multi-choice quiz exploring spices and brewing traditions with real-time scoring.
- **Reflex Rush:** Millisecond reaction test measuring extraction timing.
- **Table QR Duel:** Local multiplayer room generation with 4-digit code and QR sharing for friends.
- **Ludo & Uno:** Classic board and card games built directly in React without heavy canvas engines.

PHASE 7: KITCHEN DISPLAY SYSTEM (KDS) & REAL-TIME TRACKER

Phase Goal: Bridge the gap between the diner's table and the kitchen operations.

7.1 Order Lifecycle State Machine

When an order is submitted, the Kitchen Display System sounds an audio notification and adds the ticket to the kitchen pipeline. Kitchen cooks update status from "Pending" to "Preparing", which ticks an elapsed preparation clock. Once marked "Ready", the customer screen updates with an animated collection alert.

PHASE 8: ADMINISTRATION, ANALYTICS & FRANCHISE MANAGEMENT

Phase Goal: Provide owners with a bird's-eye view of operations and outlet profiles.

8.1 Admin Dashboard Capabilities

- **Multi-Restaurant Hub:** Add, edit, switch, or import/export restaurant outlet profiles and menus via JSON.
- **Live Menu Manager:** Edit dish prices, change categories, upload photos, and toggle item out-of-stock in 1 tap.
- **Game Lounge Manager:** Enable or disable specific games per outlet based on venue style.
- **Analytics & Funnel:** Real-time sales, order preparation duration averages, and lucky draw claims.

PHASE 9: FUTURE REDESIGN & PRODUCTION DEPLOYMENT GUIDE

Phase Goal: Checklist for developers to maintain or customize the codebase.

9.1 How to Redesign the UI in the Future

Because styles utilize Tailwind utility tokens in `src/index.css` and feature components are decoupled:

- **Changing Color Palette:** Update Tailwind accent classes (e.g. replace `amber-500` with `sky-500` or `emerald-500`).
- **Replacing a Component:** Modify only the targeted feature folder (e.g. `src/features/menu/components/MenuItemCard.tsx`).
- **Connecting Real SQL/Firebase:** Replace `localStorage` calls in `CafeContext.tsx` with Firestore or REST endpoints.

9.2 Build Verification Commands

PRODUCTION VERIFICATION

```
npm run lint        # Verifies TypeScript strict typing (zero errors)
npm run build       # Compiles optimized production bundle into dist/
npm run preview     # Runs production server locally on port 3000
```